

Trabajo de Investigación N° 2
Generadores de Analizadores Léxicos

FLEX

Fast Lexical Analyzer Generator

Junio de 2026

Índice

Índice	2
1. Introducción	3
2. Conceptos previos	3
2.1. El analizador léxico en el proceso de compilación	3
2.2. ¿Qué es un token?	4
2.3. ¿Qué son las expresiones regulares?	4
2.4. ¿Qué es un autómata finito?	4
3. ¿Qué es Flex?	5
3.1. ¿Para qué se usa?	5
3.2. Historia	5
3.3. Estado actual	6
4. ¿Cómo funciona Flex por dentro?	6
Paso 1: de la expresión regular al AFN	6
Paso 2: del AFN al AFD	7
Paso 3: minimización	7
Resultado: tablas en código C	7
4.1. Dos reglas que aplica al reconocer	7
5. ¿Cómo se usa Flex?	7
5.1. El archivo de especificación (.l)	7
5.2. El flujo completo	8
5.3. Ejemplo completo: mini-lenguaje aritmético	8
5.4. Variables que Flex pone a disposición	8
5.5. Qué produce Flex con ese archivo	9
5.6. Ejecución de ejemplo	9
6. Comparación con herramientas similares	9
6.1. ¿Cuándo conviene usar Flex?	10
6.2. ¿Cuándo conviene usar otra herramienta?	10
7. Conclusión	10
8. Bibliografía	11

1. Introducción

Cuando alguien escribe un programa en cualquier lenguaje —Python, Java, C, lo que sea— el compilador o intérprete necesita leer ese texto y **entender qué significa cada parte**. Antes de analizar la gramática o la lógica del programa, hay una tarea más básica: **identificar de qué tipo es cada pedazo de texto**. ¿Esto es un número? ¿Una palabra reservada como "if"? ¿Un símbolo como "+"? Esa tarea se llama **análisis léxico** y la hace el **analizador léxico** (también llamado *scanner* o *lexer*).

Escribir un analizador léxico a mano es tedioso y propenso a errores: habría que programar, carácter a carácter, toda la lógica de reconocimiento. Ahí es donde entra **Flex**: en lugar de escribir ese código manualmente, el programador le **describe al analizador qué patrones quiere reconocer**, y Flex genera el código C automáticamente.

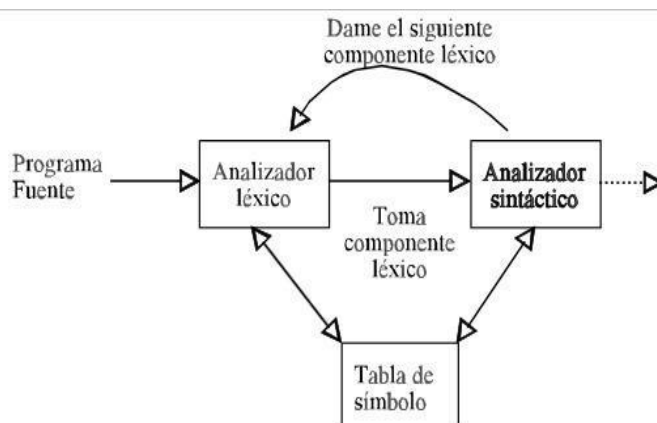
Este informe explica qué es Flex, cómo funciona por dentro, cómo se usa con un ejemplo concreto y en qué se diferencia de otras herramientas similares.

2. Conceptos previos

Para entender Flex hay tres ideas clave. No hace falta ser experto en ninguna; alcanza con tener una imagen clara de cada una.

2.1. El analizador léxico en el proceso de compilación

El siguiente diagrama muestra cómo se relaciona el analizador léxico con el resto del compilador. El analizador léxico no trabaja solo: el **analizador sintáctico** le va pidiendo tokens de a uno a medida que los necesita. Cada vez que necesita el siguiente, llama al léxico, que lee caracteres del programa fuente hasta formar un token y se lo entrega. Ambos actualizan la **tabla de símbolos**, donde se registran los identificadores del programa y sus atributos.



Este bloque **"Analizador léxico"** es exactamente el código C que Flex genera automáticamente. La función **yylex()** que Flex produce es la que responde a ese

"dame el siguiente componente léxico": cada vez que el sintáctico la llama, el scanner lee la entrada y devuelve el próximo token.

2.2. ¿Qué es un token?

Un **token** es la unidad mínima con significado dentro de un lenguaje. Pensalo como las **piezas del rompecabezas**: el texto del programa es el rompecabezas entero, y el analizador léxico lo parte en piezas individuales, cada una con una etiqueta que dice qué tipo es.

Ejemplo: dada esta línea de código:

```
resultado = 10 + velocidad
```

el analizador léxico la convierte en esta secuencia de tokens:

Token (tipo)	Lexema (texto concreto)
IDENTIFICADOR	resultado
ASIGNACION	=
NUMERO	10
OPERADOR	+
IDENTIFICADOR	velocidad

Cada token tiene dos partes: el **tipo** (la categoría, como "NUMERO" o "IDENTIFICADOR") y el **lexema** (el texto exacto que se leyó, como "10" o "velocidad").

2.3. ¿Qué son las expresiones regulares?

Una **expresión regular** es una forma compacta de describir un patrón de texto. Por ejemplo:

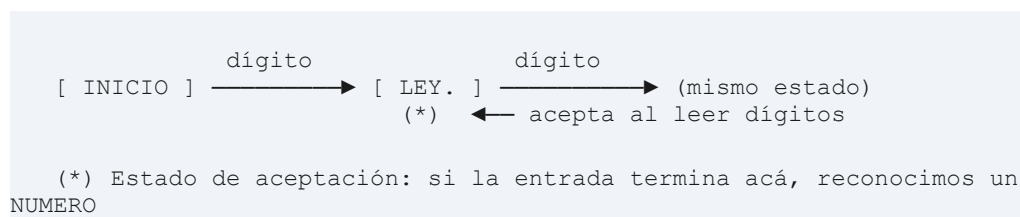
- `[0-9]+` → uno o más dígitos (describe números enteros como 3, 42, 100)
- `[a-zA-Z_][a-zA-Z0-9_]*` → una letra o guion bajo, seguidos de letras, dígitos o guiones (describe identificadores como velocidad, x1, _aux)
- `"if"|"while"|"for"` → exactamente la palabra if, while o for

Son como **plantillas de texto**: en lugar de listar cada posible número o identificador, se describe el patrón que todos ellos siguen. En Flex, cada regla del analizador se escribe como una expresión regular.

2.4. ¿Qué es un autómata finito?

Un **autómata finito** es un modelo matemático que describe una máquina muy simple: tiene un conjunto de **estados**, lee símbolos de entrada uno a uno y, según el estado actual y el símbolo leído, **pasa al siguiente estado**. Algunos estados son de **aceptación**: cuando la máquina termina de leer y está en uno de ellos, significa que reconoció el patrón.

El siguiente diagrama muestra un autómata muy simple que reconoce números enteros (uno o más dígitos):



Lo importante: internamente, Flex **convierte cada expresión regular en un autómata**. El programador no tiene que saber construirlo; lo hace Flex automáticamente.

3. ¿Qué es Flex?

Flex (*Fast Lexical Analyzer Generator*, "generador rápido de analizadores léxicos") es una herramienta que **genera código C de forma automática** a partir de una especificación de patrones escrita en expresiones regulares. El código generado es el analizador léxico completo: el programador no tiene que escribirlo.

En una frase: **vos describís qué querés reconocer, Flex escribe el código que lo reconoce.**

3.1. ¿Para qué se usa?

- **Construir compiladores e intérpretes:** Flex genera la primera etapa de cualquier compilador, el analizador léxico.
- **Procesar archivos de configuración:** leer y validar formatos de texto estructurado.
- **Filtros y conversores de texto:** transformar un formato de texto en otro.
- **Junto con Bison/Yacc:** Flex produce los tokens, Bison los organiza según la gramática del lenguaje. Es la dupla clásica para construir lenguajes completos en C.

3.2. Historia

Flex no surgió de la nada. Para entenderlo conviene conocer su antecesor:

Año	Qué pasó
1975	Mike Lesk y Eric Schmidt crean Lex en Bell Laboratories. Se convierte en herramienta estándar de Unix. Curiosidad: ese Eric Schmidt fue luego CEO de Google (2001–2011).
1979	Lex se incorpora como estándar en Unix V7. La dupla Lex + Yacc se vuelve la forma canónica de construir compiladores en Unix.
~1987	Lex era software propietario. Vern Paxson lo reescribe desde cero en C desde el Lawrence Berkeley National Laboratory y lo publica como

Año	Qué pasó
	software libre: nace Flex. El nombre "Fast" refleja que genera scanners más veloces que Lex.
2017	Se publica Flex 2.6.4, la última versión estable. Su madurez no es abandono: es una herramienta terminada que hace exactamente lo que tiene que hacer.

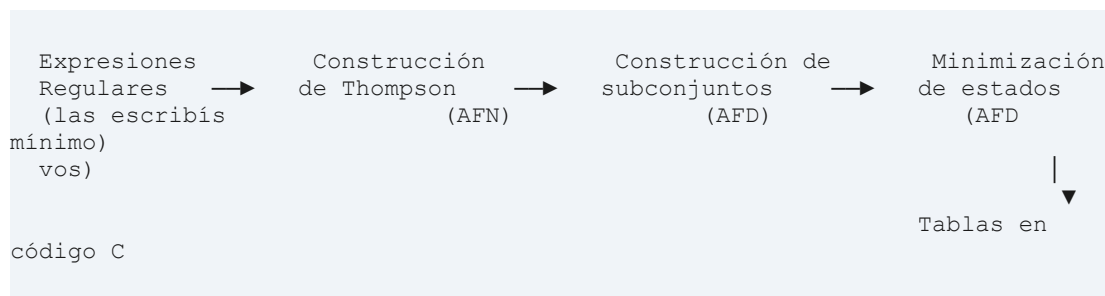
Dato interesante: **Vern Paxson**, el autor de Flex, se convirtió luego en un referente mundial de la seguridad informática. Creó **Zeek** (antes llamado Bro), uno de los sistemas de detección de intrusiones de red más usados en el mundo. Y **Van Jacobson**, quien diseñó las tablas rápidas que hacen a Flex veloz, es uno de los padres del control de congestión de TCP/IP.

3.3. Estado actual

Aspecto	Detalle
Última versión estable	2.6.4 (mayo de 2017)
Lenguaje que genera	C y C++
Licencia	BSD (software libre y gratuito)
Repositorio	github.com/westes/flex
Pareja habitual	GNU Bison (generador de analizadores sintácticos)

4. ¿Cómo funciona Flex por dentro?

Cuando Flex recibe tu especificación de patrones, no simplemente "copia" las expresiones regulares al código. Realiza una cadena de transformaciones matemáticas para construir el reconocedor más eficiente posible. Esta es la cadena:



Paso 1: de la expresión regular al AFN

Flex convierte cada expresión regular en un **Autómata Finito No Determinista (AFN)** usando la llamada construcción de Thompson. Un AFN puede tener varias flechas salientes desde un mismo estado con el mismo símbolo (es "no

determinista"). Es fácil de construir pero costoso de ejecutar porque hay que explorar varios caminos a la vez.

Paso 2: del AFN al AFD

El AFN se convierte en un **Autómata Finito Determinista (AFD)** mediante la construcción de subconjuntos. En un AFD, desde cada estado y para cada símbolo posible existe exactamente **una sola flecha**. Esto lo hace directo y eficiente de ejecutar: leer un carácter, seguir la única flecha, repetir.

Paso 3: minimización

Se fusionan los estados del AFD que son equivalentes (que reconocen exactamente las mismas cadenas desde ese punto). El resultado es el **AFD más pequeño posible**: menos estados significa tablas más compactas y un scanner más rápido.

Resultado: tablas en código C

El AFD mínimo se representa como **tablas numéricas** en el código C generado (`yy_nxt`, `yy_accept`, etc.). Un motor genérico las recorre sin necesidad de escribir un `if/else` por cada caso. Por eso Flex es rápido: todo el trabajo de pensar los estados lo hace al generar; ejecutar es solo seguir números en una tabla.

Un detalle clave: Flex hace todo esto para **todas las reglas combinadas**, no una por una. El resultado es un único AFD que reconoce números, identificadores, operadores y cualquier otro token **simultáneamente en una sola pasada** por el texto.

4.1. Dos reglas que aplica al reconocer

- **Maximal munch** (token más largo): siempre elige el token más largo posible. Si lee "index", no se detiene en "i"; consume toda la palabra.
- **Prioridad por orden**: si dos reglas coinciden con el mismo texto, gana la que aparece primero. Así, la palabra "if" se reconoce como palabra clave y no como identificador genérico.

5. ¿Cómo se usa Flex?

5.1. El archivo de especificación (.l)

El programador escribe un archivo con extensión `.l` dividido en tres secciones separadas por `%%`:

```
SECCIÓN 1: DEFINICIONES
(atajos para expresiones regulares; ej: DIGITO [0-9])

%%

SECCIÓN 2: REGLAS
(cada línea es: patrón { acción })

%%
```

SECCIÓN 3: CÓDIGO DE USUARIO
(función main y cualquier función auxiliar)

5.2. El flujo completo

```
archivo.l  →  [ flex ]  →  lex.yy.c  →  [ gcc ]  →  scanner  
(tus reglas)  (generador)  (código C  (compilador)  (ejecutable  
del autómata)                               final)
```

5.3. Ejemplo completo: mini-lenguaje aritmético

El siguiente archivo Flex reconoce números, identificadores y los operadores básicos + - * / = ():

```
/* lexer.l - analizador léxico para un mini-lenguaje aritmético */

%{
#include <stdio.h>
%}

/* SECCIÓN 1: definiciones (atajos) */
DIGITO    [0-9]
LETRA     [a-zA-Z_]

%%

/* SECCIÓN 2: reglas */
{DIGITO}+      { printf("NUMERO(%s)\n",      yytext); }
{LETRA}{LETRA}|{DIGITO}* { printf("IDENTIFICADOR(%s)\n", yytext); }
"+"           { printf("MAS\n"); }
"-"           { printf("MENOS\n"); }
"*"           { printf("POR\n"); }
"/"           { printf("DIV\n"); }
"="           { printf("ASIGNACION\n"); }
"("           { printf("PAR_IZQ\n"); }
")"           { printf("PAR_DER\n"); }
[ \t\n]+      { /* espacios: se ignoran */ }
.             { printf("DESCONOCIDO(%s)\n",  yytext); }

%%

/* SECCIÓN 3: código de usuario */
int main(void) {
    yylex(); /* yylex() es la función que Flex genera */
    return 0;
}
int yywrap(void) { return 1; }
```

5.4. Variables que Flex pone a disposición

Variable / función	Qué hace
yylex()	Función principal del scanner. Al llamarla, lee la entrada y devuelve el próximo token.
yytext	Puntero al texto exacto que acaba de reconocerse (el lexema). Ej.: si reconoció "42", yytext apunta a "42".
yylen	Longitud del texto reconocido.
yywrap()	Se llama al llegar al fin de archivo. Devolver 1 indica que no hay más entrada.

5.5. Qué produce Flex con ese archivo

1. **Parsea las definiciones:** entiende que "DIGITO" equivale a [0-9] y "LETRA" a [a-zA-Z_].
2. **Construye el autómata:** aplica la cadena $ER \rightarrow AFN \rightarrow AFD \rightarrow$ minimización para todas las reglas juntas.
3. **Genera lex.yy.c:** un archivo C con las tablas del autómata y la función yylex().
4. **El usuario compila:** `gcc lex.yy.c -o scanner` y obtiene el ejecutable.

5.6. Ejecución de ejemplo

Con la entrada:

```
resultado = 10 + velocidad
```

el scanner imprime:

```
IDENTIFICADOR(resultado)
ASIGNACION
NUMERO(10)
MAS
IDENTIFICADOR(velocidad)
```

6. Comparación con herramientas similares

Flex no es la única herramienta que genera analizadores léxicos. Existen varias alternativas, cada una orientada a un lenguaje de programación o caso de uso diferente. La siguiente tabla resume las más conocidas:

Herramienta	Año	Genera	Lo más importante
Lex	1975	C	El original, de Bell Labs. Propietario. Hoy casi en desuso, pero es la base histórica de Flex.

Herramienta	Año	Genera	Lo más importante
Flex	1987	C / C++	Libre (BSD), más rápido que Lex. Estándar de facto para proyectos en C. La herramienta de este informe.
JFlex	1998	Java	El equivalente de Flex para Java. Tiene soporte Unicode nativo, algo que Flex no tiene.
PLY	2001	Python	Lex y Yacc escritos en Python. No requiere compilación, es muy legible. Ideal para prototipado rápido.
ANTLR	1992	Java, Python, C#, JS...	Combina léxico y sintáctico en una sola herramienta. Muy usado en la industria. Más complejo pero más completo.
re2c	1994	C / C++	Genera el código más eficiente posible. No usa tablas; emite código directo. Usado en PHP, MariaDB y CPython.

6.1. ¿Cuándo conviene usar Flex?

- **Si el proyecto es en C o C++:** Flex es la opción natural, con décadas de documentación y ejemplos.
- **Para aprender cómo funcionan los compiladores:** la correspondencia directa entre expresión regular y autómata hace a Flex ideal para entender la teoría.
- **En la dupla con GNU Bison:** si además de analizar tokens se quiere construir un lenguaje completo, Flex + Bison es el camino más documentado.

6.2. ¿Cuándo conviene usar otra herramienta?

- **Python → PLY o ANTLR:** no requieren compilación intermedia y se integran naturalmente al ecosistema Python.
- **Java → JFlex:** soporte Unicode nativo y mejor integración con el ecosistema Java.
- **Rendimiento extremo en producción → re2c:** genera código más rápido que Flex porque evita las tablas en favor de código directo optimizado.
- **Léxico + sintáctico juntos → ANTLR:** permite definir toda la gramática del lenguaje en un único archivo.

7. Conclusión

Flex resuelve un problema concreto: **automatizar la escritura de analizadores léxicos**. En lugar de programar a mano cientos de líneas de código que lean

caracteres y detecten patrones, el programador describe los patrones con expresiones regulares en un archivo simple, y Flex genera el código C por él.

Por dentro, Flex aplica una cadena de transformaciones matemáticas precisas (expresión regular $\rightarrow$ AFN $\rightarrow$ AFD $\rightarrow$ código con tablas) que producen el reconocedor más eficiente posible. No es magia: es teoría de autómatas aplicada directamente.

Con más de 35 años de existencia y una última versión del 2017, Flex no es una herramienta nueva. Es una herramienta **terminada**: hace exactamente lo que tiene que hacer, de forma confiable, y la industria y la academia siguen usándola y enseñándola porque sus fundamentos —la idea de describir patrones con expresiones regulares y dejar que la máquina construya el reconocedor— **siguen siendo los correctos**.

8. Bibliografía

5. [Flex \(lexical analyzer generator\) — Wikipedia \(en inglés\)](#). Consultado: junio de 2026.
6. [Lex \(software\) — Wikipedia \(en inglés\)](#). Consultado: junio de 2026.
7. [Flex \(Fast Lexical Analyzer Generator\) — GeeksforGeeks](#). Consultado: junio de 2026.
8. Lesk, M. E. y Schmidt, E. (1975). *Lex — A Lexical Analyzer Generator*. Bell Laboratories, Computing Science Technical Report #39. Disponible en: <https://www.cs.utexas.edu/~novak/lexpaper.htm>
9. Manual oficial de Flex. Disponible en: <https://westes.github.io/flex/manual/>
10. Aho, A. V., Lam, M. S., Sethi, R. y Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools* (2.a ed.). Addison-Wesley.